

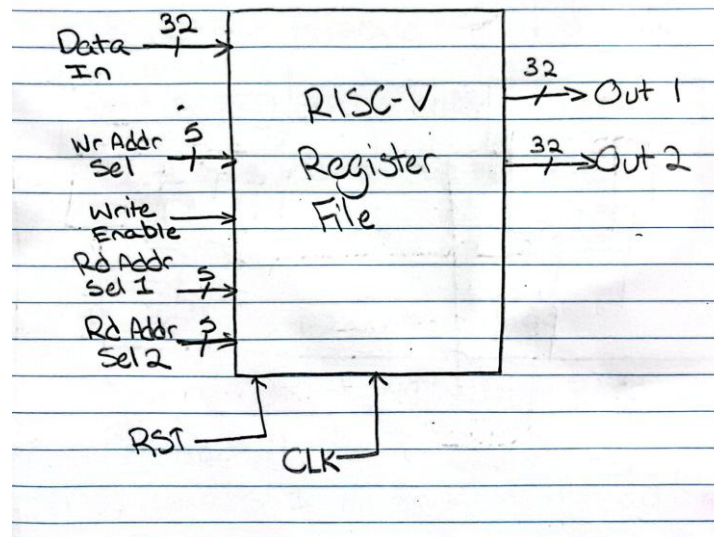
CprE 381, Computer Organization and Assembly-Level Programming

Lab 2 Report

Student Name _____ Justin Adrowski _____

Submit a typeset pdf version of this on Canvas by the due date. Refer to the highlighted language in the lab document for the context of the following questions.

[Part 3 (a)] Draw the interface description (i.e., the “symbol” or high-level blackbox) for the RISC-V register file. Which ports do you think are necessary, and how wide (in bits) do they need to be?



I indicate port widths in the diagram

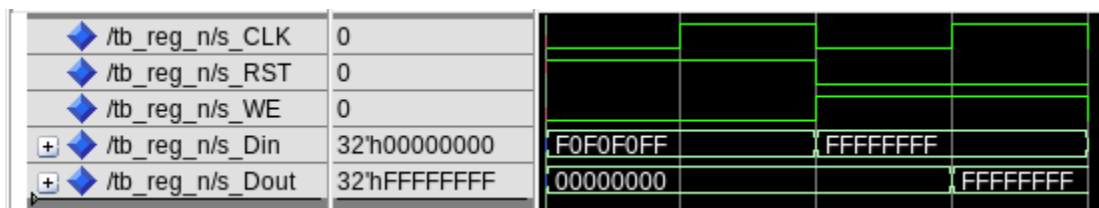
[Part 3 (b)] Create an N-bit register using this flip-flop as your basis.

```

5 library IEEE;
6 use IEEE.std_logic_1164.all;
7
8 entity reg_n is
9     generic(WIDTH : integer);
10    port(i_CLK : in std_logic;
11         i_RST : in std_logic;
12         i_WE : in std_logic;
13         i_Din : in std_logic_vector(WIDTH-1 downto 0);
14         o_Dout : out std_logic_vector(WIDTH-1 downto 0));
15 end reg_n;
16
17 architecture mixed of reg_n is
18
19     component dffg is
20         port(i_CLK : in std_logic; -- Clock input
21              i_RST : in std_logic; -- Reset input
22              i_WE : in std_logic; -- Write enable input
23              i_D : in std_logic; -- Data value input
24              o_Q : out std_logic); -- Data value output
25     end component;
26
27 begin
28     G_Nbit_DFF: for i in 0 to WIDTH - 1 generate
29         DFFi: dffg
30             port map(
31                 i_CLK => i_CLK,
32                 i_RST => i_RST,
33                 i_WE => i_WE,
34                 i_D => i_Din(i),
35                 o_Q => o_Dout(i)
36             );
37     end generate G_Nbit_DFF;
38 end mixed;

```

[Part 3 (c)] Waveform.

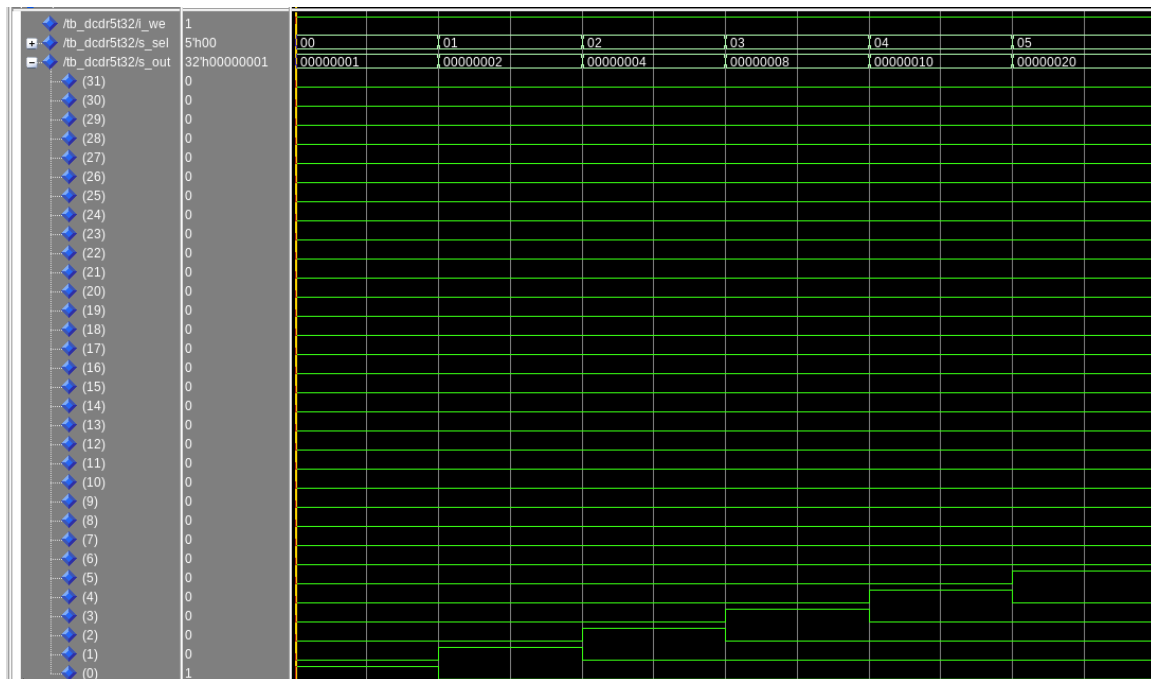


The output only changes once the write_enable line goes high, the reset is low, and the clock low-to-high edge occurs, showing that it passes functionality.

[Part 3 (d)] What type of decoder would be required by the RISC-V register file and why?

A 5-to-32 decoder since we need access to 32 registers for RISC-V, and need the address bit-field to only be five bits long.

[Part 3 (e)] **Waveform.**



The output of the decoder is always only 1 or no high bits, and the high bit always corresponds to the index of the select input, so it works.

[Part 3 (f)] **In your write-up, describe and defend the design you intend on implementing for the next part.**

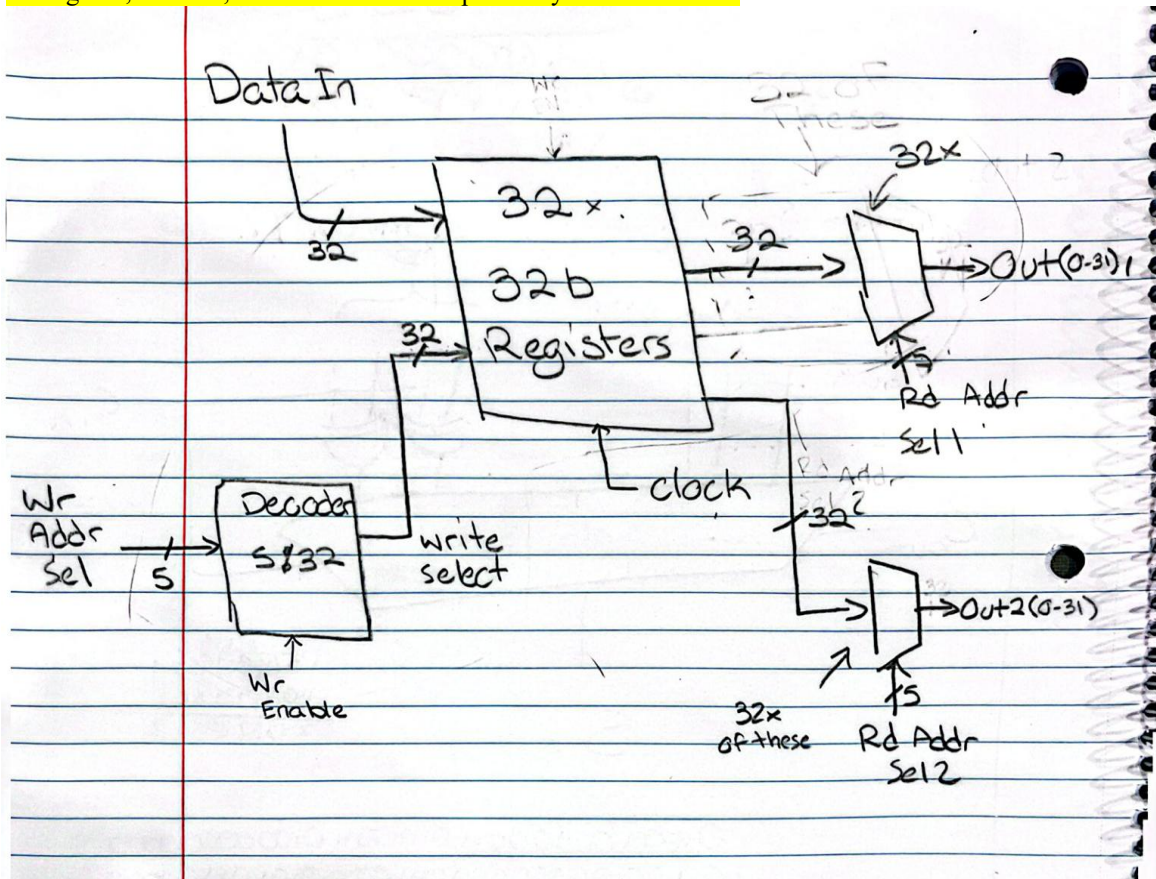
For my 32 bit 32-to-1 mux design. I made three entities. First, I augmented the 2-to-1 mux design from the previous lab to be 32-to-1. Then, based on the architecture of my mux, I realized I had to utilized a 32x32 array object as an input if I wanted to correctly implement an N-bit 32-to-1 mux. So, I did get some help from online to learn how to go this, and made another file called mux_arr.vhd, where I declare package mux_arr as a type vector array with a natural range of std_logic_vectors. Lastly, I put these together in mux32t1_n.vhd, where I declare a new input, i_array to hold all the values. Then in the final mapping, within an for loop from 0 to 31, iterating i, I generate an i-th 32-bit wide bits_for_mux signal. Each bits_for_mux contains the ith bit of each read register. Next, within the same i-th iteration of the main loop, for j registers (will be 32), I map the ith bit of the j-th register to the j-th bit of bits_for_mux. Finally, I instantiate an i-th mux32-1 gate, where I map the i-th bits_for_mux to the i_inputs of the mux gate. This occurs 32 times until each mux would output the 32 bits of the selected register given the i_sel value that goes into each mux instance.

[Part 3 (g)] **Waveform.**

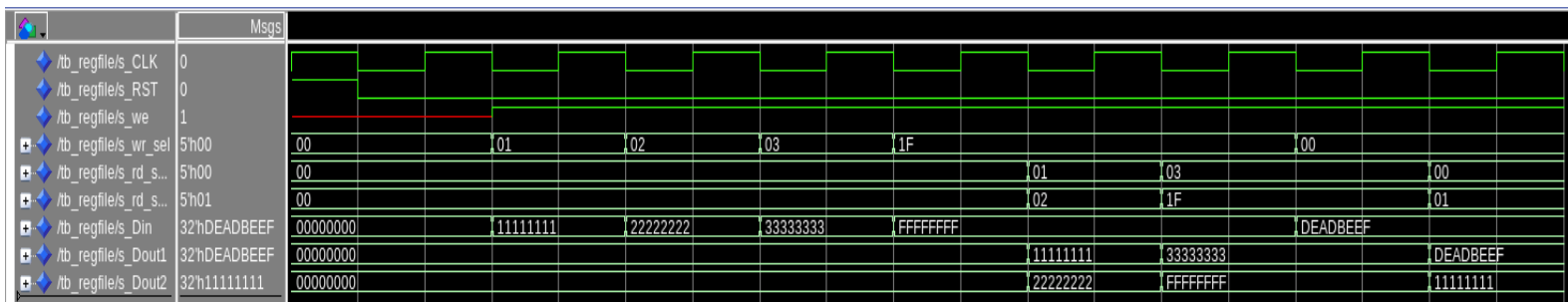
		Msgs								
			00	01	11	12	13			
/tb_mux32t1_n/s_sel		5'h1F	{0000001F}	{0000001E}	{0000001D}	{0000001C}	{0000001B}	{0000001A}	{00000019}	{00000018}
/tb_mux32t1_n/s_in		{32'h0000001F}...	0000001F							
+ (31)		32'h0000001F	0000001E							
+ (30)		32'h0000001E	0000001D							
+ (29)		32'h0000001D	0000001C							
+ (28)		32'h0000001C	0000001B							
+ (27)		32'h0000001B	0000001A							
+ (26)		32'h0000001A	00000019							
+ (25)		32'h00000019	00000018							
+ (24)		32'h00000018	00000017							
+ (23)		32'h00000017	00000016							
+ (22)		32'h00000016	00000015							
+ (21)		32'h00000015	00000014							
+ (20)		32'h00000014	00000013							
+ (19)		32'h00000013	00000012							
+ (18)		32'h00000012	00000011							
+ (17)		32'h00000011	00000010							
+ (16)		32'h00000010	0000000F							
+ (15)		32'h0000000F	0000000E							
+ (14)		32'h0000000E	0000000D							
+ (13)		32'h0000000D	0000000C							
+ (12)		32'h0000000C	0000000B							
+ (11)		32'h0000000B	0000000A							
+ (10)		32'h0000000A	00000009							
+ (9)		32'h00000009	00000008							
+ (8)		32'h00000008	00000007							
+ (7)		32'h00000007	00000006							
+ (6)		32'h00000006	00000005							
+ (5)		32'h00000005	00000004							
+ (4)		32'h00000004	00000003							
+ (3)		32'h00000003	00000002							
+ (2)		32'h00000002	00000001							
+ (1)		32'h00000001	00000000							
+ (0)		32'h00000000	00000000	00000001	00000011	00000012	00000013			
/tb_mux32t1_n/s_out		32'h0000001F	00000000	00000001	00000011	00000012	00000013			

As seen from the middle wave, s_in, I instantiated each registers value to be the register number itself. The select input is at the top, so whatever number is at the top, the same number should appear at the bottom, and as seen from the wave, this is what happened, so it worked.

[Part 3 (h)] Draw a (simplified) schematic (i.e., components within the high-level blackbox) for the RISC-V register file, using the same top-level interface ports as in your solution describe above and using only the register, decoder, and mux VHDL components you have created.

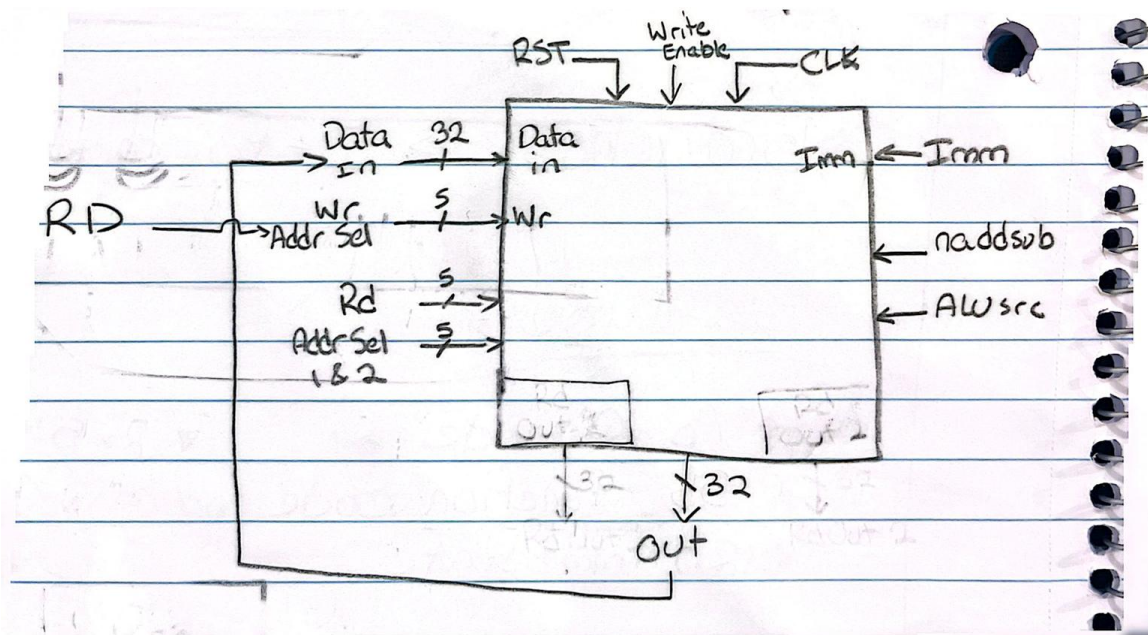


[Part 3 (i)] Waveform.

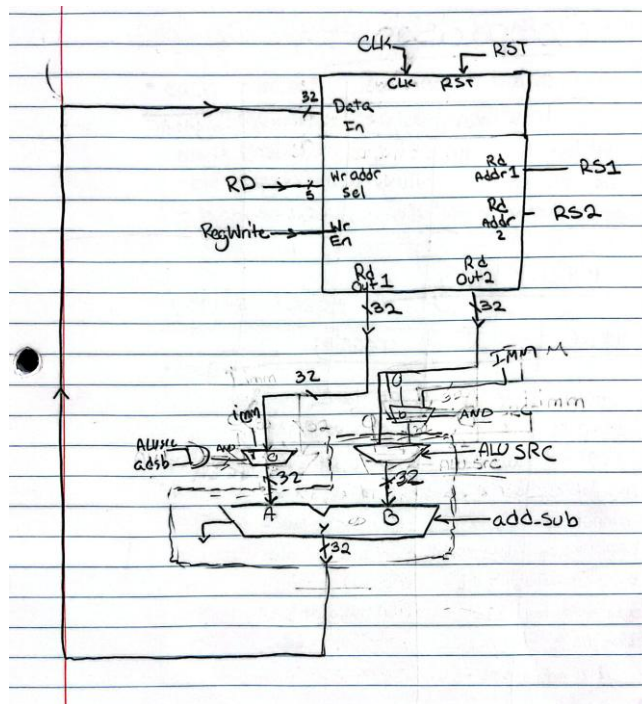


As seen from the waveform, in the first 5 clock cycles, data is written to registers 01, 02, 03, & 1F. Then, in the next 4 clock cycles, these registers are selected in both read1, and read2, where the value is displayed in Dout1 or Dout2. The previously written data matches the later read data, so the register file works.

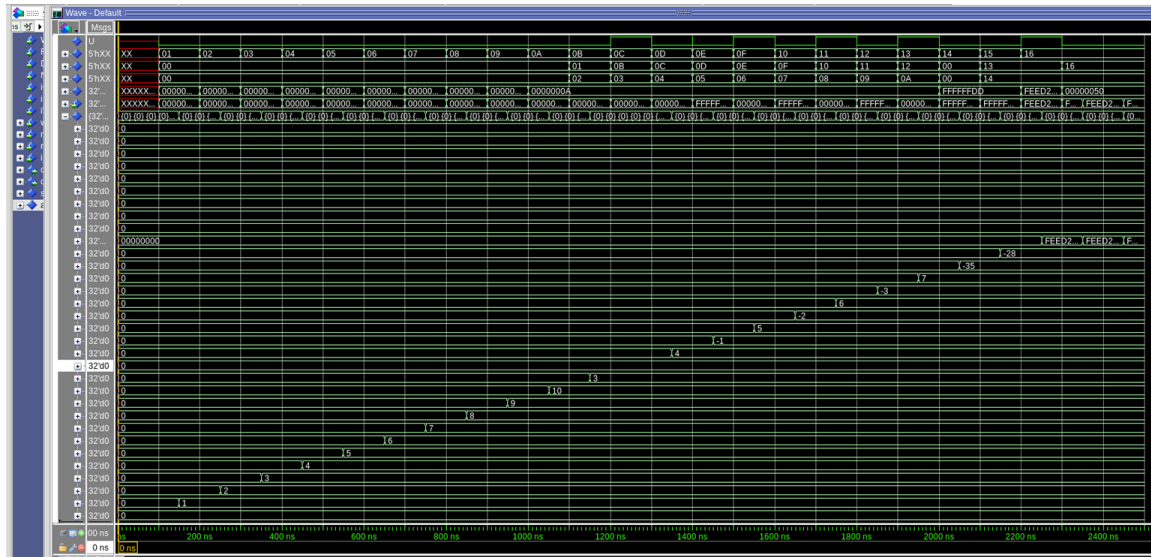
[Part 4 (b)] Draw a symbol for this RISC-V-like datapath.



[Part 4 (c)] Draw a schematic of the simplified RISC-V processor datapath consisting only of the component described in part (a) and the register file from problem (1).



[Part 4 (d)] Include in your report waveform screenshots that demonstrate your properly functioning design. Annotate what the final state of the register file should be and provide a mapping of the registers used in your drawing to the register names.



Following the RISC-V Assembly shown in the lab doc, the function is to load 1-10 into registers 1 through 10, then rotate between adding and subtracting sequential register values from one all the way up until 10. This results in the following register values, $x_{11} = 3$, $x_{12} = 0$, $x_{13} = 4$, $x_{14} = -1$, $x_{15} = 5$, $x_{16} = -2$, $x_{17} = 6$, $x_{18} = -3$, $x_{19} = 7$. Then the next instruction performs a store -35 in x_{20} , then this -35 is added to x_{19} , and stored in x_{21} , so $x_{21} = -28$. Finally the last two instructions store FEED2050 in x_{22} . All of these values end up being seen in the registers when reading the intermediate signal of the input into the mux32t1_n entity, so the module performed correctly.

[Part 5 (a)] Read through the mem.vhd file, and based on your understanding of the VHDL implementation, provide a 2-3 sentence description of each of the individual ports (both generic and regular).

DATAWIDTH $\Rightarrow$ defines the size of the data being written to/read from memory

ADDR.WIDTH $\Rightarrow$ BC PC-Relative Addressing, 2^{32} registers can be accessed w/ only 10 bits

BYTE.WIDTH $\Rightarrow$ 1 byte = 8 bits

clk $\Rightarrow$ Clock signal for synchronizing

addr $\Rightarrow$ used to determine address to read to/ write from

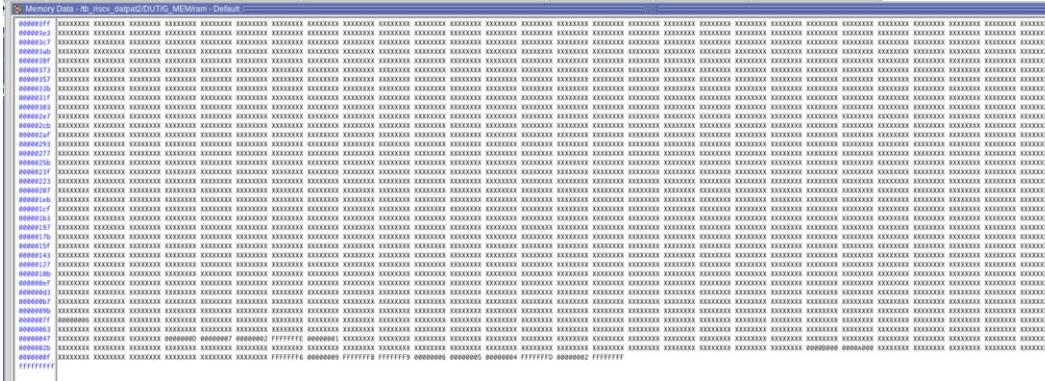
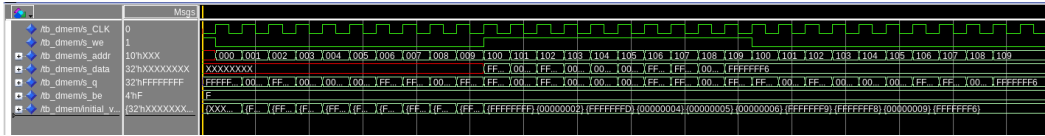
data $\Rightarrow$ bus for input write data

be $\Rightarrow$ used to determine byte of mem to access w/ each word

we $\Rightarrow$ write enable

q $\Rightarrow$ read data output, as soon as addr is set, the data at the addr index of the ram array is assigned to q

[Part 5 (c)] Waveforms and hex file.



The testbench correctly loaded the first ten location values into the next ten starting at 0x100

[Part 6 (a)] What are the RISC-V instructions that require some value to be sign extended? What are the RISC-V instructions that require some value to be zero extended? What are the RISC-V instructions that require some value to be appended with zero?

a) I-Type instructions

- * ADDI, SLTI, SLTIU
- * ANDI, ORI, XORI
- * SLUI, SRAI, SRAI
- * LW, LH, LB

S-Type Instructions

- * SW, SH, SB

B-type Instructions

- * BEQ, BNE, BLT, BGE, BLTU, BGEU

J-type Instr

- * JAL (20 bit immediate, sign ext)

Zero-extend Instr

- * SLTIU - 12b imm zero extended to compare as unsigned

Append w/ zeros (shifted left)

- * LUI - 20b imm, append 12 zeros after shift left 12b
- * AUXPC - Same as LUI, but add to PC 1st

12-bit imm, sign extend to 32b

[Part 6 (b)] what are the different “extender” components that would be required by a RISC-V processor implementation (i.e., number of bits of input and sign vs zero extension vs zero appending)?

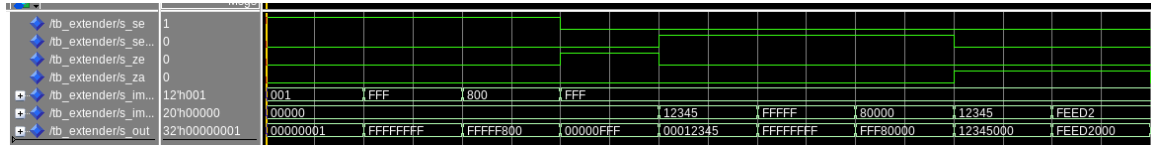
12-bit sign-extend immediate → 12 bit input, sign extension of 20 bits to 32 bit integer

20-bit sign-extend immediate → 20 bit input, sign extension of 12 bits to 32 bit integer

12-bit zero extend immediate → 12-bit input, zero-extension of 20 bits to 32 bit integer

20-bit zero append immediate → 20-bit input, zero append of 12 bits to 32 bit integer

[Part 6 (d)] **Waveform.**



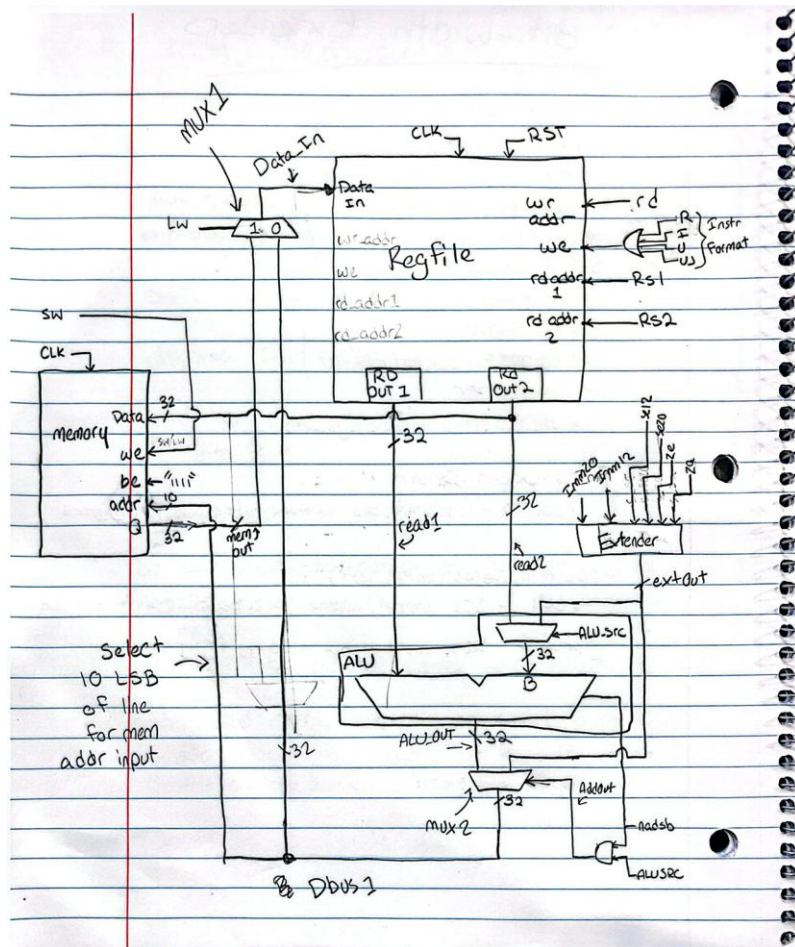
The extender has 6 inputs, four select bits for 12-bit sign/zero extension, zero append, or 20-bit sign extend, and two bus inputs for 12-bit or 20-bit immediates.

As see in the output variable, the input is extended correctly according to the corresponding input. Additionally, a hierarchy is followed as $i_se12 \rightarrow i_se20 \rightarrow i_ze \rightarrow i_za$

[Part 7 (a)] what control signals will need to be added to the simple processor from part 2? How do these control signals correspond to the ports on the mem.vhd component analyzed in part 3?

a store word signal was added, along with load word and all the extension signals. The store word signal is the only one that directly interacts with the memory block as the we input to allow data from the read out 2 to be stored into memory at the location specified by $rs1+imm$

[Part 7 (b)] Draw a schematic of a simplified RISC-V processor consisting only of the base components used in part 2, the extender component described in part 4, and the data memory from part 3.



[illegible][illegible][illegible]